

Python in Plain Eng... · [Follow publication](#)

Top 50 PySpark and Spark Interview Questions

9 min read · Jun 10, 2025



Dr. Sanjay Kumar

Follow



Listen



Share



More



Open in app ↗



Medium



◆ 1. Why Spark processing is faster than MapReduce jobs?

- Spark processes data in-memory using Resilient Distributed Datasets (RDDs), avoiding repeated disk I/O unlike MapReduce.

- Spark uses **Directed Acyclic Graph (DAG)** execution for optimized task planning, while MapReduce relies on rigid stage-by-stage processing.
- Result: **Spark can be up to 100x faster** than MapReduce for iterative and interactive workloads.

◆ 2. Spark vs MapReduce

Feature	Spark	MapReduce
Processing	In-memory	Disk-based
Performance	Faster (up to 100x)	Slower due to I/O overhead
Ease of Use	High (APIs in Scala, Python)	Low (Java-centric, verbose)
Data Structures	RDD, <u>DataFrame</u> , Dataset	Key-value pairs
Real-time support	Yes (Spark Streaming)	No

◆ 3. Why Spark was Developed?

- To overcome the **limitations of Hadoop MapReduce**:
- High disk I/O
- Complex code structure
- Lack of support for streaming, ML, graph
- Spark offers a **unified engine** for batch, streaming, ML, and graph processing with in-memory speed.

◆ 4. What is Spark?

- Apache Spark is an open-source **distributed computing engine** designed for big data processing, capable of:
- **Batch processing**
- **Stream processing**
- **Machine learning**
- **Graph analytics**
- It provides APIs in **Scala, Python (PySpark), Java, R**.

◆ 5. What is PySpark?

- PySpark is the **Python API for Apache Spark**, enabling Python developers to write Spark applications using Python syntax.
- It supports DataFrames, SQL, MLlib, GraphX, and Streaming modules.

◆ 6. What are the characteristics of PySpark?

- Ease of use for Python developers
- Lazy evaluation for transformations
- Fault tolerance via RDD lineage
- In-memory processing
- Support for MLlib, Spark SQL, Streaming

◆ 7. Features of Spark and Advantages & Disadvantages of PySpark

Features of Spark:

- In-memory computation
- DAG execution
- Unified API for batch/streaming/ML
- Supports multiple languages

Advantages of PySpark:

- Simplifies distributed computing in Python
- Easy integration with Pandas, NumPy

Disadvantages:

- Python serialization overhead
- Limited Python library support in cluster mode vs Scala

◆ 8. What is Spark Driver?

- The **driver program** is the main control process in a Spark application:
- Maintains **SparkContext**

- Translates user code into DAG
- **Distributes tasks** to executors
- Collects results and tracks execution

◆ **9. PySpark Architecture**

- **Driver Program:** Hosts SparkContext, controls execution
- **Cluster Manager:** YARN, Mesos, or Standalone
- **Executors:** Worker nodes executing tasks and storing data
- **Task DAG:** Logical execution plan created by the driver

◆ **10. PySpark Modules & Packages**

- **Spark SQL:** Structured data processing
- **MLlib:** Machine learning
- **GraphFrames/GraphX:** Graph processing
- **Spark Streaming:** Real-time data
- **Spark Core:** Underlying execution engine

◆ **11. Spark Components**

- **Spark Core:** Base engine for memory management, fault recovery, task scheduling.
- **Spark SQL:** Module for structured data and SQL queries using DataFrames.
- **Spark Streaming:** Real-time stream processing.
- **MLlib:** Machine learning library.
- **GraphX:** Graph computation engine.
- **SparkR & PySpark:** R and Python APIs.

◆ **12. What is SparkContext?**

- SparkContext is the **entry point** to the Spark Core.

It:

- Connects to the **cluster manager**
- Allocates resources
- Sends **RDD operations**
- Manages jobs and stages

Example:

```
from pyspark import SparkContext
```

```
sc = SparkContext("local", "AppName")
```

◆ 13. What is SparkSession Explained?

- SparkSession is the **entry point** for Spark 2.0+ for both Spark Core and Spark SQL.
- It replaces SQLContext, HiveContext, and integrates SparkContext.

Example:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("MyApp").getOrCreate()
```

◆ 14. SparkContext vs SparkSession

Feature	<u>SparkContext</u>	<u>SparkSession</u>
Use	Core Spark (RDD API)	Unified entry point (RDD + SQL)
Introduced in	Spark 1.x	Spark 2.x+
Capabilities	RDD creation, job management	<u>DataFrames</u> , SQL, UDFs, etc.
Replacement	–	Supersedes <u>SQLContext</u> & <u>HiveContext</u>

◆ 15. Repartition() vs Coalesce()?

Operation	<u>repartition()</u>	<u>coalesce()</u>
Function	Increases/reshuffles partitions	Decreases partitions (no shuffle)
Cost	Expensive (causes full shuffle)	Cheap (no shuffle unless needed)
Use Case	When increasing partitions	When reducing partitions

◆ 16. Difference between Cache and Persist?

Feature	<u>cache()</u>	<u>persist()</u>
Storage	Stores in memory only	Stores in memory + optional disk
Usage	Shortcut for <u>persist(MEMORY_ONLY)</u>	Custom storage level (DISK_ONLY, MEMORY_AND_DISK)
Flexibility	Less	More (choose persistence strategy)

◆ 17. What is Unpersist?

- Used to **remove** RDD/DataFrame from cache or persistence.
- Frees memory/storage resources.

`df.unpersist()`

◆ 18. Difference between Broadcast Variable and Accumulator Variable

Feature	Broadcast Variable	Accumulator
Purpose	Share read-only variables efficiently	Perform write-only aggregations
Usage	Avoid sending copies to every executor	Sum/count operations across workers
Mutable?	No	Yes (but only through actions)

◆ 19. What is Shuffling in Spark?

- **Shuffling** is the process of **redistributing** data across partitions.
- Happens during wide transformations (e.g., `groupByKey`, `reduceByKey`)
- Costly due to disk I/O, network transfer, and serialization.

◆ 20. GroupByKey vs reduceByKey vs aggregateByKey vs sortBy vs sortByKey

Method	Use Case	Shuffling	Performance
<code>groupByKey()</code>	Groups all values for each key	Yes	Slow & memory-heavy
<code>reduceByKey()</code>	Combines values per key (with reduce)	Yes	Efficient (combiner used)
<code>aggregateByKey()</code>	Aggregates values with initial value	Yes	More flexible
<code>sortBy()</code>	Sorts RDD/DataFrame by any column	Yes	Good for DataFrames
<code>sortByKey()</code>	Sorts RDD by key	Yes	Only for (K, V) RDDs

◆ 21. What is RDD?

- RDD (Resilient Distributed Dataset) is the **core data structure** in Spark.
- It is an **immutable, distributed collection of objects** that can be processed in parallel.
- Offers fault tolerance via **lineage** and **lazy evaluation**.

◆ 22. How to Create RDD?

You can create RDDs using:

1. Parallelize a local collection:

```
data = [1, 2, 3]
```

```
rdd = sc.parallelize(data)
```

1. Read from an external source:

```
rdd = sc.textFile("file.txt")
```

◆ 23. Types of RDD

- **Parallelized Collections:** From existing local collections.
- **HDFS-backed RDDs:** From data in HDFS or other storage (CSV, JSON).
- **Transformed RDDs:** Created using transformations (map, filter, etc.).

◆ 24. When to Use RDDs?

- When **fine-grained control** over transformations is needed.
- For **complex ETL pipelines** with custom logic.

- When **type-safety** and **performance tuning** are critical (Scala/Java).
- For operations not supported by DataFrames (e.g., low-level transformations).

◆ 25. RDD Operations: Transformations and Actions

Transformations (lazy):

- `map()`, `flatMap()`, `filter()`, `groupByKey()`, `reduceByKey()`, `join()`

Actions (trigger execution):

- `collect()`, `count()`, `take()`, `saveAsTextFile()`, `reduce()`

◆ 26. map vs flatMap vs filter

Operation	Description	Output Format
<u><code>map()</code></u>	Applies a function to each element	1:1 mapping (same # elements)
<u><code>flatMap()</code></u>	Flattens after applying a function	<u>1:N</u> mapping (flattened output)
<u><code>filter()</code></u>	Keeps elements that meet condition	Subset of original

Example:

```
rdd.map(lambda x: [x,x]).collect() # [[1,1], [2,2]]
```

```
rdd.flatMap(lambda x: [x,x]).collect() # [1,1,2,2]
```

```
rdd.filter(lambda x: x > 2).collect()
```

◆ 27. collect vs collectAsList vs select()

Function	Used For	Context
<u><code>collect()</code></u>	Brings full RDD/ <u>DataFrame</u> to driver	RDD/ <u>DataFrame</u>
<u><code>collectAsList()</code></u>	Converts to Python list (RDD)	RDD only
<u><code>select()</code></u>	Selects columns in <u>DataFrame</u>	<u>DataFrame</u>

◆ 28. Why DataFrame is faster than RDD?

- Uses **Catalyst Optimizer** for logical and physical plan optimization.
- Benefits from **Tungsten engine** (memory management and code generation).
- Performs **columnar storage and vectorized execution**.
- Supports **predicate pushdown** and **partition pruning**.

◆ 29. RDDs vs. DataFrames vs. Datasets

Feature	RDD	DataFrame	Dataset (Scala/Java)
Type Safety	Yes	No	Yes
Performance	Low	High (Catalyst + Tungsten)	High
API Level	Low-level (functional)	High-level (SQL-like)	Typed API (only Scala/Java)
Language Support	All (Python, Java, Scala)	All	Scala, Java only

◆ 30. Pivot and Unpivot a Spark DataFrame

Pivot:

```
df.groupBy("Category").pivot("Month").sum("Revenue")
```

Unpivot (using stack):

```
from pyspark.sql.functions import expr
```

```
df.select("Category", expr("stack(3, 'Jan', Jan, 'Feb', Feb, 'Mar', Mar) as (Month, Revenue)"))
```

◆ 31. What is Spark Schema?

- A **schema** defines the structure of a DataFrame (columns, names, types).
- Spark infers schema automatically or it can be explicitly defined using **StructType**.

Example:

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
```

```
schema = StructType([
```

```
StructField("name", StringType(), True),
```

```
StructField("age", IntegerType(), True)
```

```
])
```

```
df = spark.read.csv("people.csv", schema=schema)
```

◆ 32. GroupBy clause

- Similar to SQL's GROUP BY, used to **group data** based on one or more columns for aggregation.

Example:

```
df.groupBy("department").agg({"salary": "avg"}).show()
```

◆ 33. What is Spark SQL DataFrame?

- A **DataFrame** in Spark SQL is a distributed collection of data organized into named columns.
- It supports:
- SQL queries
- Lazy evaluation
- Optimized execution plans via Catalyst
- Integration with Hive, JSON, Parquet, etc.

◆ 34. Why DataFrame?

- Simplifies **data manipulation** with a high-level API.
- More **efficient** than RDDs due to Catalyst and Tungsten optimizations.
- Supports **SQL-like operations**.
- Enables **interoperability** between Spark and external systems.

◆ 35. Is PySpark faster than pandas?

Scenario	Winner	Reason
Small data (fits in memory)	pandas	Lightweight, in-memory
Big data (distributed)	<u>PySpark</u>	Runs on multiple nodes, scalable

- PySpark is better for **large-scale distributed processing**, while pandas is more performant for **small local data**.

◆ 36. What is DAG and lineage graph, RDD lineage?

- DAG (Directed Acyclic Graph):** Represents the sequence of computations (transformations → actions).
- Lineage Graph:** Tracks dependencies between RDDs.
- Enables **fault tolerance**: Spark can recompute lost data from its lineage.

◆ 37. What is paired RDD?

- An RDD consisting of **key-value pairs**, useful for operations like `reduceByKey`, `groupByKey`.

Example:

```
rdd = sc.parallelize([("a", 1), ("b", 2), ("a", 3)])
```

```
rdd.reduceByKey(lambda x, y: x + y).collect() # Output: [('a', 4), ('b', 2)]
```

◆ 38. What is skewness?

- Data skew** occurs when some partitions receive significantly more data than others.
- This leads to **unbalanced workload** and degraded performance due to straggler tasks.

◆ 39. How to mitigate skewed data?

- Techniques include:
- Salting:** Add a random prefix/suffix to keys before group operations.
- Repartitioning:** Redistribute data using `repartition()` or `coalesce()`.

- **Broadcast joins:** Avoid shuffle by broadcasting smaller tables.
- **Skew join optimization:** Use Spark's built-in skew join hints.

◆ 40. Optimization Techniques in Spark

- **Persist/Caching** intermediate data
- **Avoid shuffles** (prefer reduceByKey over groupByKey)
- **Use broadcast variables** for small lookup data
- **Partition tuning** (repartition, coalesce)
- **Predicate pushdown** (filter early)
- **Efficient formats** (Parquet/ORC over CSV/JSON)
- **Enable Tungsten and Catalyst** optimizations

◆ 41. How to read CSV File Using delimiter ','?

```
df = spark.read.option("delimiter", ",").csv("data.csv", header=True, inferSchema=True)
```

- `.option("delimiter", ",")` explicitly defines the delimiter
- `header=True` treats the first row as column names
- `inferSchema=True` automatically infers data types

◆ 42. What is Star Schema & Snowflake Schema? Differentiate them

Feature	Star Schema	Snowflake Schema
Structure	Central fact table with denormalized dimension tables	Fact table with normalized dimension tables
Complexity	Simple	Complex (more joins)
Query Speed	Faster due to fewer joins	Slower due to multiple joins
Use Case	OLAP systems	Data warehouses with high normalization

◆ 43. What is Data Skewness?

- **Skewness** refers to an **uneven distribution** of data across partitions or keys.

- Causes:
- Unequal key distribution in joins or aggregations
- Unbalanced partitions
- Effects:
- Straggler tasks
- Memory pressure
- Increased job runtime

◆ **44. What is Catalyst Optimizer?**

- Catalyst is Spark SQL's query optimization engine.
- Performs:
- Logical plan generation
- Logical optimization
- Physical plan selection
- Code generation (via Tungsten)
- Enables high performance and extensibility in Spark SQL and DataFrames.

◆ **45. Explain Serialization and Deserialization?**

- **Serialization:** Converting an object into a byte stream to store or transfer it.
- **Deserialization:** Reconstructing the original object from a byte stream.
- In Spark, serialization is crucial when sending data between executors or caching.

◆ **46. What are PySpark serializers?**

1. JsonSerializer:

- Default in Spark (uses Java serialization)
- Slower and verbose

1. KryoSerializer:

- Faster and more compact
- Requires class registration for custom objects

```
conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
```

◆ 47. Salting Techniques

- Used to handle **data skew** in joins or aggregations.
- Technique:
 - Add a **random prefix/suffix** to skewed keys
 - Distribute data across partitions
 - After processing, remove the salt

Example:

```
from random import randint
```

```
rdd.map(lambda x: (str(randint(0, 5)) + "_" + x[0], x[1]))
```

◆ 48. Explain MapPartition in Spark

- mapPartitions() is similar to map(), but operates on **entire partitions** instead of individual elements.
- More efficient when:
 - You want to reuse resources (e.g., DB connections)
 - You want to reduce overhead of function calls

Example:

```
rdd.mapPartitions(lambda partition: [sum(partition)])
```

◆ 49. How to track failed Jobs in Spark?

- Use **Spark UI** (<http://<driver-node>:4040>)
- View stages, jobs, DAGs, and failure reasons
- **Logs** (stdout/stderr or YARN logs)
- **Task metrics** from monitoring tools (e.g., Ganglia, Prometheus)

- Retry policies and **speculative execution** help recover from failures

◆ 50. What is Broadcast Join?

- A **join optimization** where the smaller dataset is **broadcasted** to all executor nodes.
- Avoids shuffle by keeping one dataset local.

```
df_large.join(broadcast(df_small), "id")
```

- Use when one table is **small enough to fit in memory**.

◆ 51. Deployment Mode? Cluster mode and Client Mode

Mode	Description
Client	Driver runs on the machine that submits the application
Cluster	Driver runs inside the cluster (e.g., on YARN/Mesos/Standalone manager)

- **Client Mode:** Better for development, debugging
- **Cluster Mode:** Recommended for production workloads

◆ 52. Spark Submit Command

Used to submit applications to a Spark cluster.

Example:

```
bash
```

```
CopyEdit
```

```
spark-submit \
```

```
— master yarn \
```

```
— deploy-mode cluster \
```

```
— executor-memory 4G \
```

```
— num-executors 4 \
```

my_script.py

Key options:

- — master: local, yarn, mesos
- — deploy-mode: client or cluster
- — executor-memory, — executor-cores, etc.

◆ 53. ORC vs Parquet vs CSV vs JSON

Format	Compression	Schema Support	Speed	Use Case
CSV	None	No	Slow	Simplicity, readability
JSON	Minimal	Semi-structured	Slow	Nested data, flexible
Parquet	Yes	Yes	Fast (columnar)	Best for analytical workloads
ORC	Yes	Yes	Fast	Preferred in Hive & Hadoop stack

Parquet and ORC are best for **big data analytics**, with **Parquet** widely supported in Spark.

◆ 54. Deal with Bad Data

- Options in read():

```
spark.read.option("mode", "DROPMALFORMED").csv("data.csv")
```

- Modes:
- PERMISSIVE: Default, inserts nulls
- DROPMALFORMED: Ignores bad rows
- FAILFAST: Fails immediately
- Also use:
- try-except in UDFs
- Filtering logic to drop invalid rows

◆ 55. Why Out of Memory issue occurs?

- Causes:

- Large joins/shuffles
- Too many cached datasets
- Skewed data
- Improper partitioning
- Solutions:
- Use persist() wisely
- Repartition data
- Increase executor memory
- Avoid collect() on large datasets

◆ 56. How to Remove Duplicate Rows?

```
df.dropDuplicates()
```

or drop duplicates based on specific columns:

```
df.dropDuplicates(["id", "name"])
```

◆ 57. Create SparkContext & SparkSession

```
from pyspark import SparkContext
```

```
sc = SparkContext(appName="MyApp")
```

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("MyApp").getOrCreate()
```

◆ 58. How to Create RDD

- From a collection:

```
rdd = sc.parallelize([1, 2, 3])
```

- From a file:

```
rdd = sc.textFile("file.txt")
```

◆ 59. Create (Read) Spark DataFrame from CSV, Txt, JSON, XML

CSV

```
df_csv = spark.read.option("header", True).csv("data.csv")
```

TXT (as text)

```
df_txt = spark.read.text("data.txt")
```

JSON

```
df_json = spark.read.json("data.json")
```

XML (requires spark-xml package)

```
df_xml = spark.read.format("xml").option("rowTag", "book").load("data.xml")
```

Thank you for being a part of the community

Before you go:

- Be sure to **clap** and **follow** the writer 🙌
- Follow us: [X](#) | [LinkedIn](#) | [YouTube](#) | [Newsletter](#) | [Podcast](#) | [Twitch](#)
- [Start your own free AI-powered blog on Differ](#) 🚀
- [Join our content creators community on Discord](#) 🧑
- For more content, visit [plainenglish.io](#) + [stackademic.com](#)

[Data Science](#)[Data Analysis](#)[AI](#)[Artificial Intelligence](#)[Machine Learning](#)[Follow](#)

Published in Python in Plain English

98K followers · Last published 17 hours ago

New Python content every day. Follow to join our 3.5M+ monthly readers.



Follow

Written by Dr. Sanjay Kumar

2.1K followers · 457 following

Data Science | Enterprise Architect | Data Governance & Cloud Transformation Leader | AI Engineering | AI Product | Author | Keynote Speaker

